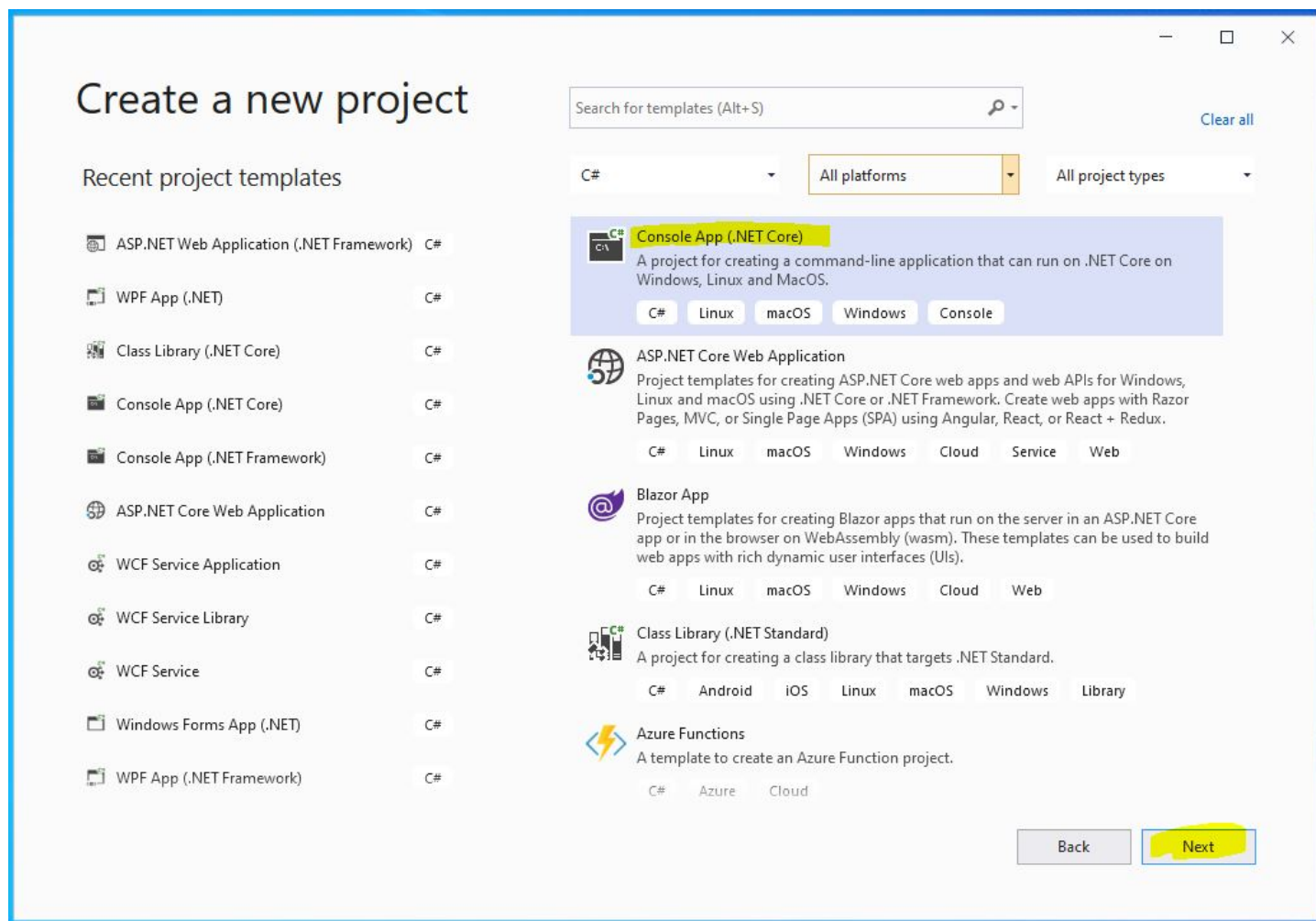


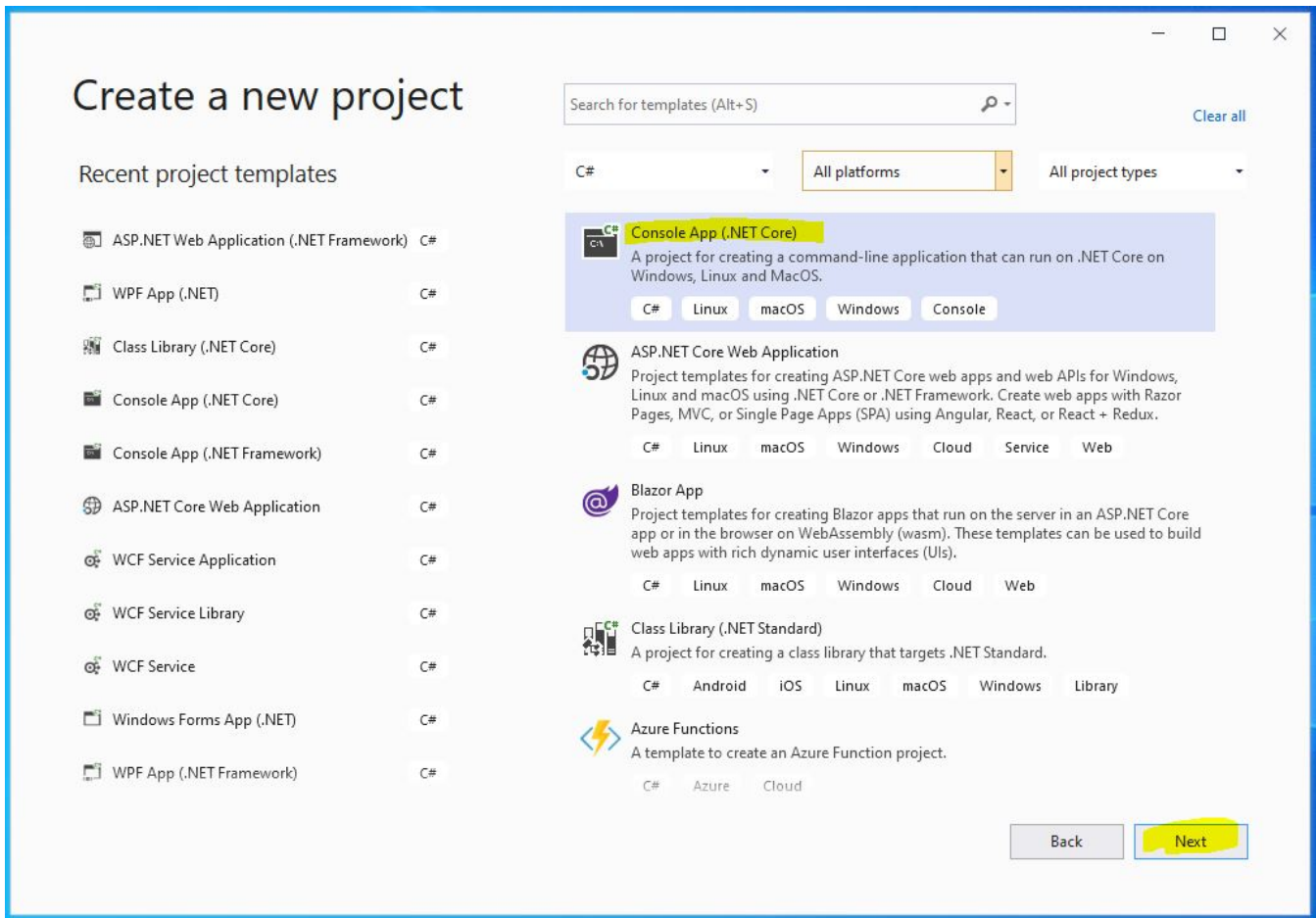
# Pierwsza Aplikacja w C# – Programujemy Kalkulator

Najwyższa pora napisać pierwszą taką powiedzmy bardziej rozszerzoną i kompletną aplikację. Myślę, że warto zacząć od takiego prostego kalkulatora.

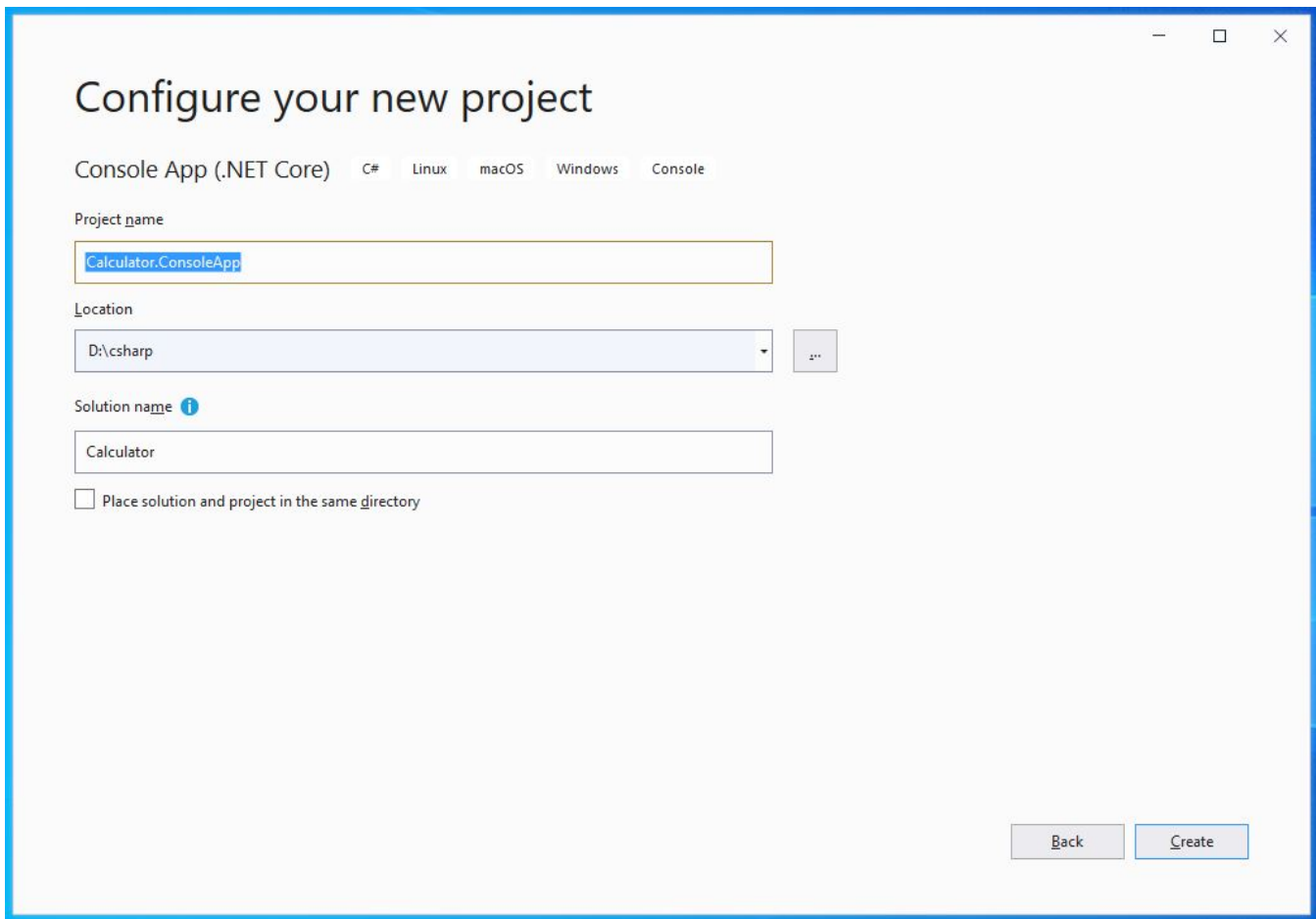
Na początek uruchom proszę visual studio i utworzymy nowy projekt konsolowy w C#.



Kliknij Create a new project.



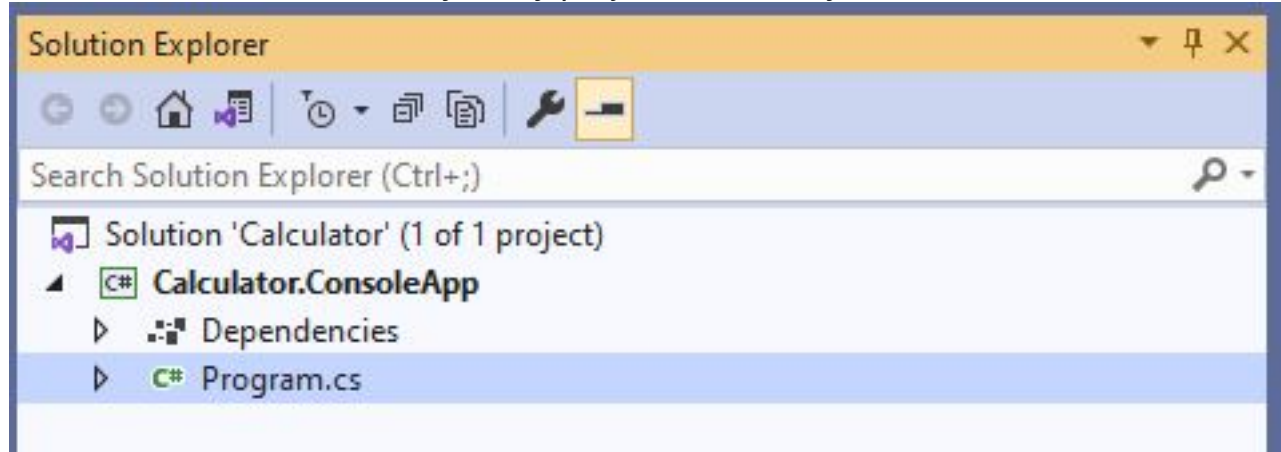
Wybierz Console App (.NET Core) i kliknij Next.



Wpisz nazwę projektu. Może to być Calculator.ConsoleApp, z kolei nazwa solucji niech będzie Calculator.

Na kolejnych lekcjach stworzymy taki kalkulator jako aplikacja desktopowa, webowa i mobilna, dlatego do nazwy projektu dodajemy przyrostek ConsoleApp tak, aby było wiadomo, że ten konkretny projekt to aplikacja konsolowa. Wszystkie te projekty będziemy tworzyć w solucji o nazwie Calculator. Kliknij create.

Po chwili zostanie utworzony nowy projekt konsolowy.



Zgodnie z wpisanymi wcześniej nazwami, została utworzona solucja Calculator oraz projekt Calculator.ConsoleApp. Kliknij 2 krotnie na plik Program.cs i usuń kod w metodzie Main. Tak wygląda w ten sposób kod naszej aplikacji:

```
namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

Zanim zaczniemy pisać kod, warto zastanowić się dokładnie nad tym, jak powinna działać nasza aplikacja. Zróbmy proste notatki, komentarze, gdzie napiszemy krok po kroku, jak nasza aplikacja będzie działać.

- Pierwszy krok to będzie wyświetlenie jakiegoś nagłówka, czyli opis aplikacji.
- Następnie wyświetlimy komunikat z prośbą o podanie 1 liczby.
- Pobierzemy liczbę od użytkownika.
- Użytkownik będzie musiał wybrać działanie/operację, jaką chce wykonać. To znaczy dodawanie, odejmowanie, mnożenie lub dzielenie, dlatego musimy w tym miejscu wyświetlić stosowny komunikat.
- To działanie przypiszemy sobie do jakiejś zmiennej.
- Prośba o podanie 2 liczby.
- Pobranie 2 liczby od użytkownika.
- Na podstawie tych danych wykonamy obliczenia.
- I na koniec wyświetlimy użytkownikowi wynik działania.

Czyli tak będzie działać nasza aplikacja.

```
namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            //2. Prośba o podanie 1 liczby
            //3. Pobranie liczby od użytkownika
            //4. Prośba o podanie działania
            //5. Pobranie wybranego działania od użytkownika
            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

Mam wszystko ładnie rozpisane, wiemy krok po kroku, jak aplikacja ma działać. To są takie nasze minimalne wymagania, zatem możemy przejść do implementacji powyższych punktów.

Na początek wyświetlenie nagłówka, użyjemy do tego statycznej metody WriteLine z klasy Console.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            //3. Pobranie liczby od użytkownika
            //4. Prośba o podanie działania
            //5. Pobranie wybranego działania od użytkownika
            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

Kolejny krok to wyświetlenie komunikatu użytkownikowi z prośbą o podanie 1 liczby:

```
using System;
namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            //4. Prośba o podanie działania
            //5. Pobranie wybranego działania od użytkownika
            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

Pobranie liczby od użytkownika. Tym razem skorzystamy ze statycznej metody ReadLine klasy Console i przypiszemy wartość do zmiennej o nazwie number1.

```
using System;
namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = Console.ReadLine();

            //4. Prośba o podanie działania
            //5. Pobranie wybranego działania od użytkownika
            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

Musimy w tym miejscu uważać na 1 rzecz, ponieważ użytkownik w konsoli może wpisać dowolną wartość, ponieważ metoda ReadLine zwraca wartość tekstową – string'a. W związku z tym w tym miejscu musimy rzutować wartość wpisaną przez użytkownika na wartość liczbową, niech to będzie int. Możemy do tego użyć metody statycznej Parse:

```
using System;
namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            //5. Pobranie wybranego działania od użytkownika
            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

To rzutowanie można zapisać lepiej, jednak na tą chwilę spróbujmy tylko zaimplementować wymagania minimum, a później wrócimy jeszcze do tego kodu.

Kolejny etap to wyświetlenie komunikatu z prośbą o podanie przez użytkownika działania, które będziemy wykonywać. To już wiesz jak zrobić. Możemy tutaj również wyświetlić dostępne operacje, które może wybrać:

```
(. . .)
            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            Console.WriteLine("Jaką operację chcesz wykonać? Możliwe
operacje to: '+', '-', '*', '/'.");

(. . .)
```

Będziemy w tym miejscu oczekiwać działania. Wybrane działanie zgodnie z kolejnym krokiem przypiszemy do zmiennej.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.");

            //5. Pobranie wybranego działania od użytkownika
            var operation = Console.ReadLine();

            //6. Prośba o podanie 2 liczby
            //7. Pobranie liczby od użytkownika
            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```

Kolejny krok to ponownie pobranie liczby od użytkownika, tym razem 2.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka – opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.");

            //5. Pobranie wybranego działania od użytkownika
            var operation = Console.ReadLine();

            //6. Prośba o podanie 2 liczby
            Console.WriteLine("Podaj proszę 2 liczbę:");

            //7. Pobranie liczby od użytkownika
            var number2 = int.Parse(Console.ReadLine());

            //8. Wykonanie obliczeń
            //9. Wyświetlenie wyniku użytkownikowi
        }
    }
}
```



Pozostaje nam teraz wykonanie obliczeń. W tej sytuacji najlepiej będziesz skorzystać z instrukcji switch, przekazując wybraną operację, czyli zmienną operation:

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka - opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.");

            //5. Pobranie wybranego działania od użytkownika
            var operation = Console.ReadLine();

            //6. Prośba o podanie 2 liczby
            Console.WriteLine("Podaj proszę 2 liczbę:");

            //7. Pobranie liczby od użytkownika
            var number2 = int.Parse(Console.ReadLine());

            //8. Wykonanie obliczeń
            var result = 0;

            switch (operation)
            {
                case "+":
                    result = number1 + number2;
                    break;
                case "-":
                    result = number1 - number2;
                    break;
                case "*":
                    result = number1 * number2;
                    break;
                case "/":
                    result = number1 / number2;
                    break;
                default:
                    throw new Exception("Wybrałeś złą operację!");
            }

            //9. Wyświetlenie wyniku użytkownikowi

        }
    }
}
```



Jeżeli użytkownik wybierze +, to wartość dodawania przypiszemy do nowej zmiennej result. Analogicznie zrobimy z pozostałymi operacjami. Jeżeli użytkownik wpisze inny znak, to wtedy zostanie rzucony wyjątek z odpowiednią wiadomością. Czyli tak może wyglądać cała instrukcja switch. Na podstawie wybranej operacji zostaną wykonane odpowiednie obliczenia. Wynik zostanie przypisany do zmiennej result. Na koniec wystarczy ten wynik za pomocą interpolacji stringów wyświetlić w konsoli użytkownikowi.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            //1. Wyświetlenie nagłówka - opis aplikacji
            Console.WriteLine("Witaj w aplikacji KALKULATOR!");

            //2. Prośba o podanie 1 liczby
            Console.WriteLine("Podaj proszę 1 liczbę:");

            //3. Pobranie liczby od użytkownika
            var number1 = int.Parse(Console.ReadLine());

            //4. Prośba o podanie działania
            Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.");

            //5. Pobranie wybranego działania od użytkownika
            var operation = Console.ReadLine();

            //6. Prośba o podanie 2 liczby
            Console.WriteLine("Podaj proszę 2 liczbę:");

            //7. Pobranie liczby od użytkownika
            var number2 = int.Parse(Console.ReadLine());

            //8. Wykonanie obliczeń
            var result = 0;

            switch (operation)
            {
                case "+":
                    result = number1 + number2;
                    break;
                case "-":
                    result = number1 - number2;
                    break;
                case "*":
                    result = number1 * number2;
                    break;
                case "/":
                    result = number1 / number2;
                    break;
                default:
                    throw new Exception("Wybrałeś złą operację!");
            }

            //9. Wyświetlenie wyniku użytkownikowi
            Console.WriteLine($"Wynik Twojego działania to: {result}.");
        }
    }
}
```

Tak może wyglądać nasza aplikacja. Wszystkie wymagania minimum, które wcześniej sobie rozpisaliśmy, zostały spełnione. Możemy teraz uruchomić aplikację (CTRL + F5) i przeprowadzić kilka testów w celu upewnienia się, że wszystko działa zgodnie z oczekiwaniami.

```
Microsoft Visual Studio Debug Console
Witaj w aplikacji KALKULATOR!
Podaj proszę 1 liczbę:
6
Jaka operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
+
Podaj proszę 2 liczbę:
4
Wynik Twojego działania to: 10.
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe (process 6956) exited wi
th code 0.
Press any key to close this window . . .
```

Jak widzisz, wszystko działa. Jednak, mimo to naszą aplikację nie jest jeszcze kompletna. Zauważ, że jeżeli użytkownik nie będzie stosował się do komunikatów i np. zamiast wybrania 1 z 4 dostępnych operacji wpisze dowolne ciąg znaków, to zostanie rzucony nieobsłużony w żaden sposób wyjątek. Przez to nasza aplikacji w takiej sytuacji co prawda wyświetli komunikat, ale zakończy działanie.

```
Microsoft Visual Studio Debug Console
Witaj w aplikacji KALKULATOR!
Podaj proszę 1 liczbę:
10
Jaka operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
test
Podaj proszę 2 liczbę:
10
Unhandled exception. System.Exception: Wybrałeś złą operację!
   at Calculator.ConsoleApp.Program.Main(String[] args) in D:\csharp\Calculator2\Calculator.ConsoleApp2\Program.cs:line
52
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe (process 12768) exited w
ith code -532462766.
Press any key to close this window . . .
```

Aby to poprawić, wystarczy cały kod, w którym spodziewamy się, że może wystąpić wyjątek wywołać wewnątrz instrukcji try catch. Jeżeli jakiś wyjątek w takiej sytuacji wystąpi, to obsłużymy go w klauzuli catch.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                //1. Wyświetlenie nagłówka - opis aplikacji
                Console.WriteLine("Witaj w aplikacji KALKULATOR!");

                //2. Prośba o podanie 1 liczby
                Console.WriteLine("Podaj proszę 1 liczbę:");

                //3. Pobranie liczby od użytkownika
```

```

var number1 = int.Parse(Console.ReadLine());

//4. Prośba o podanie działania
Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'");

//5. Pobranie wybranego działania od użytkownika
var operation = Console.ReadLine();

//6. Prośba o podanie 2 liczby
Console.WriteLine("Podaj proszę 2 liczbę:");

//7. Pobranie liczby od użytkownika
var number2 = int.Parse(Console.ReadLine());

//8. Wykonanie obliczeń
var result = 0;

switch (operation)
{
    case "+":
        result = number1 + number2;
        break;
    case "-":
        result = number1 - number2;
        break;
    case "*":
        result = number1 * number2;
        break;
    case "/":
        result = number1 / number2;
        break;
    default:
        throw new Exception("Wybrałeś złą operację!");
}

//9. Wyświetlenie wyniku użytkownikowi
Console.WriteLine($"Wynik Twojego działania to: {result}");
}
catch (Exception ex)
{
    //logowanie do pliku
    Console.WriteLine(ex.Message);
}
}
}
}

```

Oprócz tego wyświetlimy użytkownikowi ładny komunikat na temat błędu, który wystąpił bez zbędnych informacji. Nasza aplikacja nie przerwie działania, mimo błędu będzie można dalej z niej korzystać.

```

Select Microsoft Visual Studio Debug Console
Witaj w aplikacji KALKULATOR!
Podaj proszę 1 liczbę:
10
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
test
Podaj proszę 2 liczbę:
5
Wybrałeś złą operację!
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe <process 12908> exited w
ith code 0.
Press any key to close this window . . .

```

Jak widzisz tym razem, został wyświetlony komunikat bez zbędnych informacji, aplikacja nie przerwała działania.

Wszystkie założenia zostały spełnione, dlatego dobrą praktyką jest również usunięcie tych zbędnych komentarzy, tak żeby sam kod był bardziej przejrzysty i czytelny.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Witaj w aplikacji KALKULATOR!");

                Console.WriteLine("Podaj proszę 1 liczbę:");
                var number1 = int.Parse(Console.ReadLine());

                Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'");
                var operation = Console.ReadLine();

                Console.WriteLine("Podaj proszę 2 liczbę:");
                var number2 = int.Parse(Console.ReadLine());

                var result = 0;

                switch (operation)
                {
                    case "+":
                        result = number1 + number2;
                        break;
                    case "-":
                        result = number1 - number2;
                        break;
                    case "*":
                        result = number1 * number2;
                        break;
                    case "/":
                        result = number1 / number2;
                        break;
                    default:
                        throw new Exception("Wybrałeś złą operację!");
                }

                Console.WriteLine($"Wynik Twojego działania to: {result}.");
            }
            catch (Exception ex)
            {
                //logowanie do pliku
                Console.WriteLine(ex.Message);
            }
        }
    }
}
```

Zauważ, że w kilku miejscach używamy tego samego kodu. Np.

```
var number1 = int.Parse(Console.ReadLine());
var number2 = int.Parse(Console.ReadLine());
```

Czyli pobieramy wartość od użytkownika i rzutujemy ją na int'a. Warto trochę taki kod zrefaktoryzować. Najlepiej w takim przypadku utworzyć nową prywatną metodę, która zostanie użyta w 2 miejscach.

```
private static int GetInput()
{
    return int.Parse(Console.ReadLine());
}
```

Musi to być metoda statyczna, ponieważ będziemy ją wywoływać w metodzie statycznej Main. Będzie ona zwracać już przekonwertowaną na int'a wartość wpisaną przez użytkownika. Następnie możemy zamienić wywołania w metodzie Main i kod będzie wyglądał tak:

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Witaj w aplikacji KALKULATOR!");

                Console.WriteLine("Podaj proszę 1 liczbę:");
                var number1 = GetInput();

                Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'");
                var operation = Console.ReadLine();

                Console.WriteLine("Podaj proszę 2 liczbę:");
                var number2 = GetInput();

                var result = 0;
                switch (operation)
                {
                    case "+":
                        result = number1 + number2;
                        break;
                    case "-":
                        result = number1 - number2;
                        break;
                    case "*":
                        result = number1 * number2;
                        break;
                    case "/":
                        result = number1 / number2;
                        break;
                    default:
                        throw new Exception("Wybrałeś złą operację!");
                }
                Console.WriteLine($"Wynik Twojego działania to: {result}");
            }
            catch (Exception ex)
            {
                //logowanie do pliku
                Console.WriteLine(ex.Message);
            }
        }

        private static int GetInput()
        {
            return int.Parse(Console.ReadLine());
        }
    }
}
```

Aby jeszcze zwiększyć czytelność naszego kodu, możemy sobie nasze obliczenia, czyli całą instrukcję switch wywołać w osobnej metodzie.

```
private static double Calculate(double number1, double number2, string operation)
{
    switch (operation)
    {
        case "+":
            return number1 + number2;
        case "-":
            return number1 - number2;
        case "*":
            return number1 * number2;
        case "/":
            return number1 / number2;
        default:
            throw new Exception("Wybrałeś złą operację!\n");
    }
}
```

Czyli dodaliśmy nową prywatną statyczną metodę o nazwie Calculate, która na podstawie przekazanych parametrów zwraca wynik obliczeń. Nie potrzebujemy też tutaj tymczasowej zmiennej result, tylko od razu ze switcha'a zwrócimy wartość obliczeń. Jeżeli używamy return, to break jest również zbędny. Pozostaje tylko wywołać nową metodę w metodzie Main.

```
using System;

namespace Calculator.ConsoleApp
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Witaj w aplikacji KALKULATOR!");

                Console.WriteLine("Podaj proszę 1 liczbę:");
                var number1 = GetInput();

                Console.WriteLine("Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.");
                var operation = Console.ReadLine();

                Console.WriteLine("Podaj proszę 2 liczbę:");
                var number2 = GetInput();

                var result = Calculate(number1, number2, operation);

                Console.WriteLine($"Wynik Twojego działania to: {result}.");
            }
            catch (Exception ex)
            {
                //logowanie do pliku
                Console.WriteLine(ex.Message);
            }
        }

        private static int GetInput()
        {
            return int.Parse(Console.ReadLine());
        }

        private static int Calculate(int number1, int number2,
string operation)
        {
            switch (operation)
            {
                case "+":
```

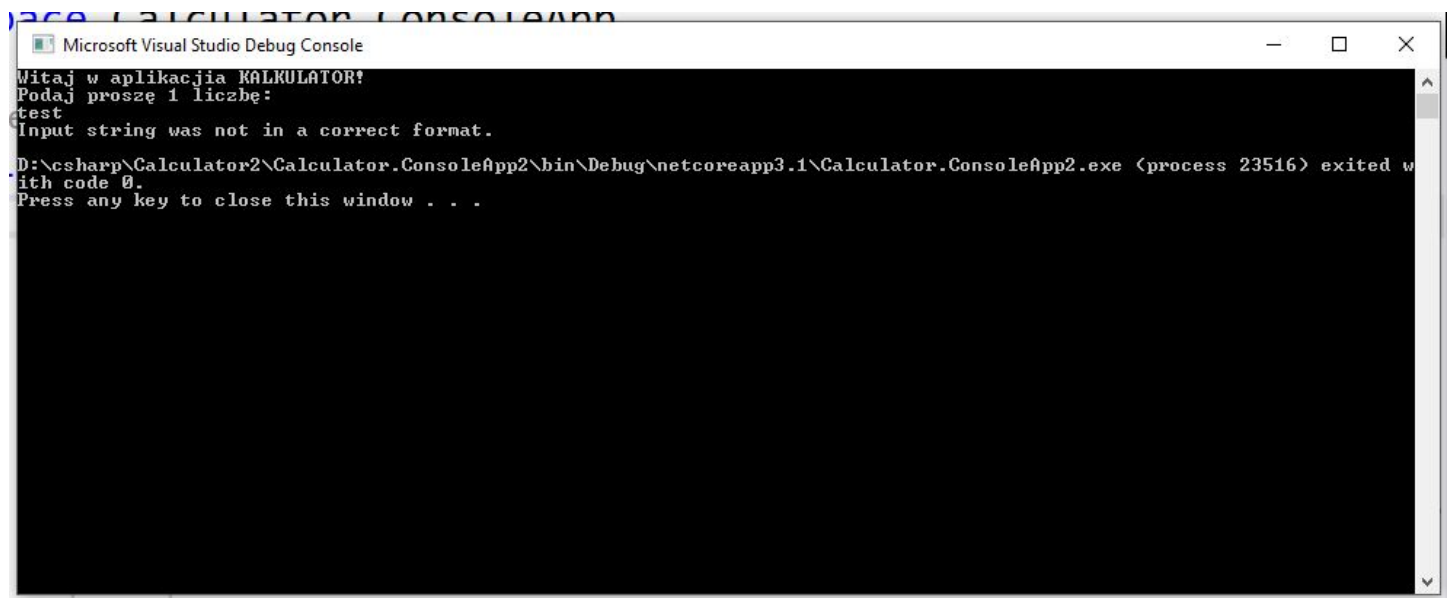
```

        return number1 + number2;
    case "-":
        return number1 - number2;
    case "*":
        return number1 * number2;
    case "/":
        return number1 / number2;
    default:
        throw new Exception("Wybrałeś złą operację!");
    }
}
}
}

```

Dzięki takim zabiegom zmniejszył się kod w metodzie Main. Dodaliśmy 2 nowe metody, które są bardziej czytelne i każda z nich odpowiada za 1 konkretną rzecz. Co również bardzo ważne, zostały odpowiednio nazwane, dzięki czemu nawet bez patrzenia do ciała tych metod, po samej nazwie możemy domyślić się, co każda z nich robi.

Nasza aplikacja dalej działa tak samo, jak wcześniej, ale możemy ją jeszcze trochę udoskonalić. Zauważ teraz, że jeżeli przy próbie podania 1 liczby, użytkownik wpisze jakąś wartość tekstową, która nie jest liczbą, to zostanie rzucony wyjątek. Co prawda mamy taką sytuację obsługowaną, wyświetlamy wtedy odpowiedni komunikat w konsoli, ale możemy to jeszcze rozwiązać w inny sposób.



W takiej sytuacji do konwertowania tekstu na liczbę zamiast `int.Parse`, lepiej użyć `int.TryParse`.

```

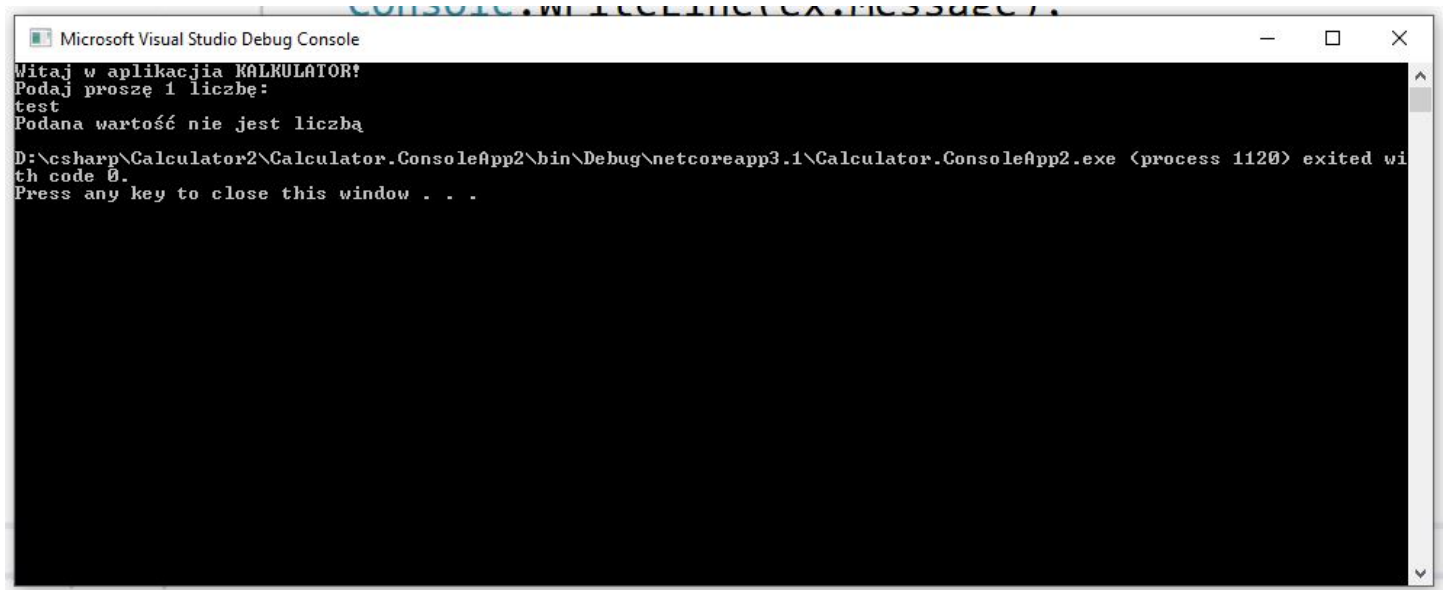
private static int GetInput()
{
    if (!int.TryParse(Console.ReadLine(), out int input))
        throw new Exception("Podana wartość nie jest liczbą");
}

```



```
    return input;  
}
```

Czyli jeżeli nie uda się przekonwertować wartości wpisanej przez użytkownika na liczbę całkowitą typu int, to zostanie rzucony wyjątek z odpowiednim komunikatem, w tym przypadku "Podana wartość nie jest liczbą." Jeżeli wpisaną wartość uda się przekonwertować na liczbę, to ta liczba zostanie zwrócona. Do metody TryParse przekazałem zmienną input z parametrem out. Zobaczmy, jak teraz zachowa się aplikacja. Zauważ, że dzięki temu, że wprowadziliśmy prywatną metodę GetInput, te zmiany wystarczyło zrobić tylko w 1 miejscu.



```
Microsoft Visual Studio Debug Console  
Witaj w aplikacji KALKULATOR!  
Podaj proszę 1 liczbę:  
test  
Podana wartość nie jest liczbą  
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe (process 1120) exited with code 0.  
Press any key to close this window . . .
```

Czyli wszystko zadziała według oczekiwań. Tylko dalej można usprawnić jej działanie. Zauważ, że teraz, jeżeli użytkownik chce wykonać jakieś obliczenia, to może to zrobić tylko 1 raz. W sytuacji, gdy chce ponownie wykonać obliczenia, to za każdym razem musi uruchamiać ponownie aplikację. Nie jest to dobre rozwiązanie. Fajnie jakby użytkownik mógł wykonać dowolną ilość obliczeń i w każdej chwili mógł z niej wyjść. Aby to zrobić, wystarczy takie obliczenia wykonywać w pętli, w naszym przypadku możemy skorzystać z pętli nieskończonej np. while:

```
while (true)  
{  
}
```

Cały kod wykonamy w takiej pętli. Użytkownik będzie mógł wykonać wiele obliczeń i wyjść z aplikacji w dowolnym momencie.

Dodatkowo po zakończeniu pętli dodałem do komunikatu znak `\n`, aby następny komunikat w konsoli był wyświetlany od nowej linii. Zwiększy to trochę przejrzystość aplikacji.

```
var result = Calculate(number1, number2, operation);
```

```
Console.WriteLine($"Wynik Twojego działania to: {result}.  
\\n");
```

```
    }  
    catch (Exception ex)  
    {  
        //logowanie do pliku  
        Console.WriteLine(ex.Message);  
    }  
}
```

```
private static int GetInput()
```

```
{  
    if (!int.TryParse(Console.ReadLine(), out int input))  
        throw new Exception("Podana wartość nie jest liczbą.\\n");  
  
    return input;  
}
```

```
private static int Calculate(int number1, int number2, string operation)
```

```
{  
    switch (operation)  
    {  
        case "+":  
            return number1 + number2;  
        case "-":  
            return number1 - number2;  
        case "*":  
            return number1 * number2;  
        case "/":  
            return number1 / number2;  
        default:  
            throw new Exception("Wybrałeś złą operację!\\n");  
    }  
}
```

```
throw new Exception("Podana wartość nie jest liczbą.  
\\n");
```

```
return input;
```

```
}
```

```
private static double Calculate(double number1, double  
number2, string operation)
```

```
{  
    switch (operation)  
    {  
        case "+":  
            return number1 + number2;  
        case "-":  
            return number1 - number2;  
        case "*":  
            return number1 * number2;  
        case "/":  
            return number1 / number2;  
        default:  
            throw new Exception("Wybrałeś złą operację!\\n");  
    }  
}
```

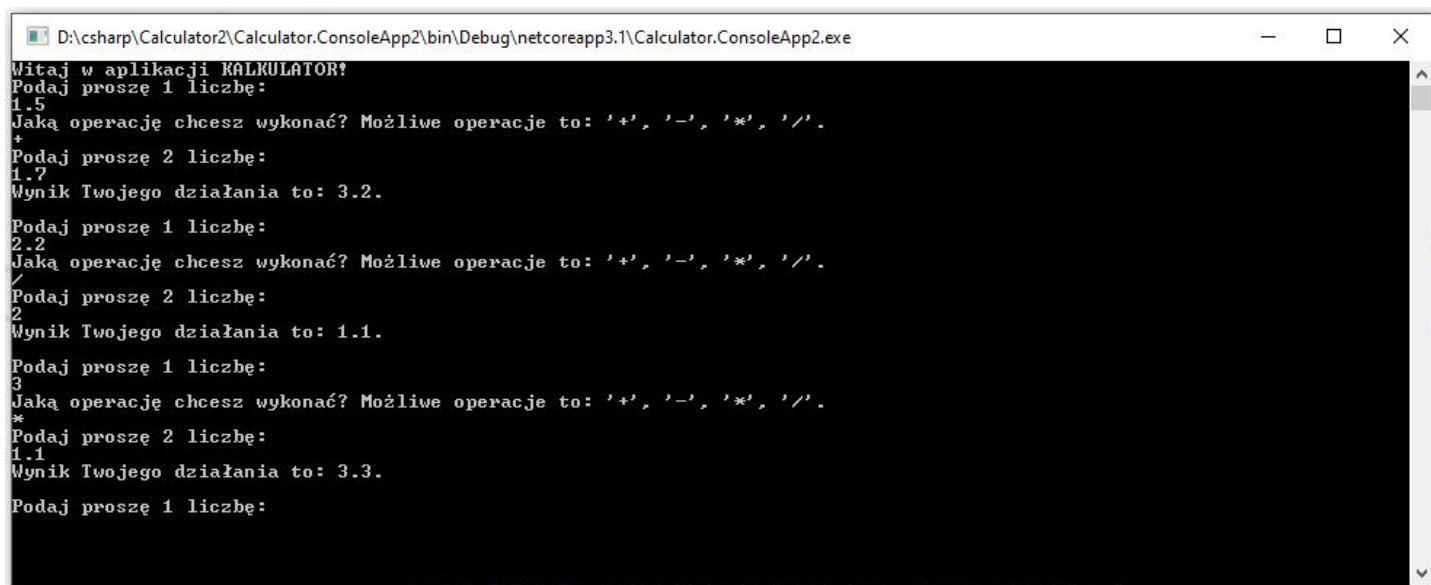
```
}
```

```
}
```

```
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe
Witaj w aplikacji KALKULATOR!
Podaj proszę 1 liczbę:
10
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
+
Podaj proszę 2 liczbę:
5
Wynik Twojego działania to: 15.
Podaj proszę 1 liczbę:
test
Podana wartość nie jest liczbą.
Podaj proszę 1 liczbę:
14
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
/
Podaj proszę 2 liczbę:
2
Wynik Twojego działania to: 7.
Podaj proszę 1 liczbę:
```

Na koniec jeszcze 1 usprawnienie, które koniecznie musimy zrobić. Zauważ, że teraz nasz kalkulator działa na int'ach, czyli liczbach całkowitych. Warto byłoby to zmienić i obsłużyć tak, aby kalkulator robił obliczenia również liczb rzeczywistych. W tym celu możemy zamiast int'a użyć double'a. Także, musimy we wszystkich miejscach zmienić te wywołania. Dodatkowo jak będziemy wyświetlać wynik, to możemy dzięki statycznej metodzie Round z klasy Math zaokrąglić go do 2 miejsc po przecinku. Tak ostatecznie będzie wyglądała nasza aplikacja Kalkulator:

Pozostaje nam teraz tylko uruchomienie aplikacji i przetestowania kilka przypadków, tak aby upewnić się, że wszystko dobrze działa.



```
D:\csharp\Calculator2\Calculator.ConsoleApp2\bin\Debug\netcoreapp3.1\Calculator.ConsoleApp2.exe
Witaj w aplikacji KALKULATOR!
Podaj proszę 1 liczbę:
1.5
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
+
Podaj proszę 2 liczbę:
1.7
Wynik Twojego działania to: 3.2.

Podaj proszę 1 liczbę:
2.2
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
-
Podaj proszę 2 liczbę:
2
Wynik Twojego działania to: 1.1.

Podaj proszę 1 liczbę:
3
Jaką operację chcesz wykonać? Możliwe operacje to: '+', '-', '*', '/'.
*
Podaj proszę 2 liczbę:
1.1
Wynik Twojego działania to: 3.3.

Podaj proszę 1 liczbę:
```

## PODSUMOWANIE

Jak widzisz, udało nam się napisać Twoją pierwszą kompletną aplikację w języku C#. Najpierw rozpisaliśmy wszystkie założenia, jakie musi spełniać ta aplikacja. Następnie je zaimplementowaliśmy i udało nam się również wprowadzić kilka nowych funkcjonalności, dzięki którym aplikacja jest jeszcze lepsza. Jeżeli chcesz, to taką aplikację możesz spokojnie jeszcze rozwijać, wprowadzać dodatkowe operacje i ćwiczyć, dzięki czemu będziesz podnosił swoje umiejętności. Pamiętaj, że jeżeli chcesz się rozwijać, musisz praktykować, musisz programować, także najlepiej pisać różne aplikacje.